



Documentação Técnica: Sistema Costs

1. Visão Geral do Projeto

O **Costs** é uma aplicação web desenvolvida em **React** focada na **gestão de projetos e serviços**. Ele permite aos usuários gerenciarem o ciclo de vida de um projeto, desde a criação e definição de orçamento até o acompanhamento dos custos de serviços.

1.1. Funcionalidades Principais

- * **Criação de Projetos:** Definir nome, orçamento e categoria.
- * **Gestão de Orçamento:** Calcular o total gasto e o saldo restante do orçamento.
- * **Gestão de Serviços:** Adicionar, listar e remover serviços associados a um projeto.
- * **CRUD Completo:** Suporte para Criar, Ler, Atualizar e Deletar (CRUD) projetos e serviços.

1.2. Objetivo Educacional

O projeto serve como uma demonstração prática de técnicas de desenvolvimento front-end modernas, incluindo:

- * Implementação de operações CRUD em uma aplicação React.
- * Organização modular e reutilização de componentes.
- * Comunicação com uma API simulada (JSON Server).
- * Gerenciamento de estado local eficiente.

1.3. Público-alvo

Esta documentação é destinada a:

- * **Desenvolvedores Intermediários** em JavaScript e React.
- * **Estudantes** que buscam entender padrões de arquitetura e desenvolvimento de mercado.

2. Tecnologias e Pré-requisitos

2.1. Tecnologias Utilizadas

Tecnologia	Descrição
React	Biblioteca JavaScript para construção de interfaces de usuário.
React Router	Biblioteca para roteamento declarativo na aplicação.
CSS Modules	Abordagem para modularizar e isolar o escopo do CSS.
JSON Server	Backend de desenvolvimento leve para simular uma API REST.
Node.js	Ambiente de execução JavaScript.
npm	Gerenciador de pacotes do Node.js.

2.2. Pré-requisitos de Instalação

Para executar o projeto localmente, você deve ter instalado:

- * Node.js (versão 14 ou superior)
- * npm (versão 6 ou superior)
- * Um navegador web moderno (Chrome, Firefox, Edge).

3. Instalação e Execução

Siga os passos abaixo para configurar e rodar a aplicação Costs em seu ambiente local.

3.1. Configuração Inicial

1º Clonar o Repositório:

```
git clone https://github.com/kayquemab/Projeto_Costs
```

```
cd costs
```

2º Instalar Dependências:

```
npm install
```

3.2. Execução do Frontend

3º Inicie a aplicação React:

```
npm start
```

A aplicação estará acessível em <http://localhost:3000>.

3.3. Execução do Backend (JSON Server)

O projeto utiliza o JSON Server para simular o backend e a persistência de dados no arquivo db.json.

1º Inicie o servidor fake:

```
npx json-server --watch db.json --port 5000
```

Os endpoints da API estarão disponíveis em <http://localhost:5000/projects>.

4. Estrutura do Projeto

A arquitetura do projeto segue uma organização modular, separando componentes por funcionalidade (project, service, form) e agrupando páginas e utilitários.

```
costs/
├─ public/
├─ src/
│   ├─ components/
│   │   ├─ form/
│   │   │   ├─ Input.jsx
│   │   │   ├─ Select.jsx
│   │   │   └─ SubmitButton.jsx
│   │   └─ layout /
│   │       ├─ Container.jsx
│   │       ├─ Navbar.jsx
│   │       └─ Footer.jsx
│   │   └─ project /
│   │       ├─ ProjectCard.jsx
│   │       └─ ProjectForm.jsx
│   │       └─ SubmitButton.jsx
│   └─ service/
│       ├─ ServiceCard.jsx
│       └─ ServiceCard.jsx
└─ pages/
    ├─ Home.jsx
    └─ Projects.jsx
```

5. Componentes Principais

Abaixo estão os componentes React mais importantes e suas propriedades (props).

5.1. ProjectForm

Propósito: Componente de formulário reutilizável para criar ou editar projetos.

Propriedade	Tipo	Descrição
<code>handleSubmit</code>	<code>function</code>	Função chamada ao submeter o formulário.
<code>btnText</code>	<code>string</code>	Texto exibido no botão de submissão.
<code>projectData</code>	<code>object</code>	(Opcional) Dados do projeto para preenchimento do formulário em modo de edição.

Exemplo de Uso:

```
<ProjectForm  
  handleSubmit={createProject}  
  btnText="Criar Projeto"  
/>
```

5.2. ProjectCard

Propósito: Componente de card para exibição resumida de um projeto na listagem.

Propriedade	Tipo	Descrição
<code>id</code>	<code>number</code>	ID único do projeto.
<code>name</code>	<code>string</code>	Nome do projeto.
<code>budget</code>	<code>number</code>	Orçamento total definido para o projeto.

Propriedade	Tipo	Descrição
<u>category</u>	<u>object</u>	Objeto contendo o nome e ID da categoria.
<u>handleRemove</u>	<u>function</u>	Função de callback para exclusão do projeto.

Exemplo de Uso:

```
<ProjectCard
```

```

  id={1}
  name="Site institucional"
  budget={5000}
  category={{ name: "Infraestrutura" }}
  handleRemove={removeProject}

```

```
/>
```

5.3. ServiceForm

Propósito: Formulário para adicionar novos serviços dentro da página de detalhes de um projeto.

Propriedade	Tipo	Descrição
<u>handleSubmit</u>	<u>function</u>	Função chamada ao submeter o formulário.
<u>btnText</u>	<u>string</u>	Texto exibido no botão de submissão.
<u>projectData</u>	<u>object</u>	Dados do projeto ao qual o serviço será associado.

5.4. ServiceCard

Propósito: Componente de card para exibir um serviço individual dentro de um projeto.

Propriedades Principais:

- * **name:** Nome do serviço.
- * **cost:** Custo do serviço.
- * **description:** Descrição detalhada do serviço.
- * **id:** ID único do serviço.
- * **handleRemove:** Função de callback para exclusão do serviço.

5.5. Componentes de Layout e Utilidade

- * **Container:** Componente de layout para centralizar e limitar o conteúdo.

```
<Container customClass="min-height">
```

```
<h1>Projetos</h1>
```

```
</Container>
```

- * **LinkButton:** Componente que funciona como um botão, mas utiliza o roteamento do React Router.

```
<LinkButton to="/newproject" text="Criar Projeto" />
```

- * **Loading:** Componente de feedback visual para indicar o carregamento de dados.

```
{loading && <Loading />}
```

6. Modelo de Dados e Endpoints

O JSON Server utiliza o arquivo db.json como sua fonte de dados. O modelo de dados principal é o de projects, que pode conter uma lista aninhada de services.

6.1. Estrutura do `db.json`

```
{  
  
  "projects": [  
    {  
      "id": 1,  
      "name": "Construção de API",  
      "budget": 12000,  
      "category": {  
        "id": "infra",  
        "name": "Infraestrutura"  
      },  
      "services": [  
        {  
          "id": 1,  
          "name": "Consultoria",  
          "cost": 3000,  
          "description": "Planejamento inicial"  
        }  
      ]  
    }  
  ]  
}
```

6.2. Fluxo de Dados e Arquitetura

O fluxo de dados segue o padrão de uma aplicação React que interage com uma API REST:

Fluxo Geral:

- 1º O usuário interage com um **Formulário** (ex: `ProjectForm`).
- 2º O evento de submissão (`handleSubmit`) é acionado.
- 3º É feita uma chamada assíncrona ao **JSON Server** (API).
- 4º Após a resposta da API, o **Estado** da aplicação é atualizado.
- 5º A atualização do estado dispara a **Renderização** dos componentes afetados.

7. Desenvolvimento e Manutenção

7.1. Testes

Recomenda-se a integração das seguintes ferramentas para garantir a qualidade do código:

- * **Jest:** Framework de testes JavaScript.
- * **React Testing Library (RTL):** Biblioteca para testar componentes React de forma que se assemelhe ao uso real.
- * **MSW (Mock Service Worker):** Para simular requisições de rede e isolar testes de integração.

7.2. Build e Deploy

Para gerar a versão de produção da aplicação:

```
npm run build
```

Sugestões de Hospedagem:

- * **Netlify**
- * **Vercel**
- * **GitHub Pages**

7.3. Guia de Contribuição

O projeto adota o padrão **Conventional Commits** para mensagens de commit, facilitando a geração automática de changelogs e a compreensão do histórico do projeto.

Tipos de Commit Aceitos:

- * **feat:** Nova funcionalidade.
- * **fix:** Correção de bug.
- * **docs:** Alterações na documentação.
- * **style:** Alterações de formatação (sem mudança na lógica).
- * **refactor:** Refatoração de código (sem correção de bug ou nova funcionalidade).

7.4. Troubleshooting Comum

Problema: A porta 5000 (usada pelo JSON Server) está ocupada. **Solução:** Utilize o comando `kill-port` para liberar a porta:

```
npx kill-port 5000
```

7.5. Histórico de Versões (Changelog)

Versão	Data	Descrição
0.1.0	(Data da primeira versão)	Versão inicial com CRUD completo de projetos e serviços e integração com JSON Server.

8. Manutenção da Documentação

É fundamental que esta documentação seja mantida e atualizada sempre que houver a adição de novos componentes, endpoints de API, ou qualquer alteração significativa na arquitetura do sistema.